Assignment

1. Build a complete button component system that combines variables, maps, mixins, and nesting. The system should support multiple variants (primary, danger, ghost) and sizes (sm, md, lg) without repeating CSS rules.
    a. Create a `$btn-variants` map: each key maps to a nested map of `bg`, `color`, and `border` values
    b. Write a `button-variant` mixin that accepts a variant name, looks it up in the map using `map.get()`, and outputs all three properties
    c. Create a `$btn-sizes` map for padding and font-size at each size
    d. Generate `.btn-primary`, `.btn-danger`, `.btn-ghost` classes using `@each` over the variants map
    e. Use nesting inside `.btn` to add `:hover` and `:disabled` states

2. Create a reusable mixin for applying responsive padding and use SCSS lists to generate margin utility classes for a card-based layout.
    a. Define a `$spacing-scale` list: `(4px, 8px, 16px, 24px, 32px, 48px)`
    b. Use `@each` with `index()` or a counter to generate `.m-1` through `.m-6` margin classes
    c. Write a `responsive-padding` mixin that accepts `$mobile` and `$desktop` padding arguments
    d. The mixin must output a `@media` query block internally — the caller should not need to write breakpoints
    e. Apply the mixin to at least two different components: `.card` and `.section`

3. Style a responsive navigation bar using SCSS nesting. The nav has a top-level bar, a logo, a list of links, and hover states. All styles must live inside a single .navbar block.
    a. Use at least 3 levels of nesting (e.g. .navbar > ul > li > a)
    b. Use the parent selector & for :hover, :focus, and :active states on links
    c. Use variables for colors, font sizes, and spacing so the theme is easy to change
    d. No styles for .navbar should be written outside of the main .navbar { } block